Project Requirements

System Analysis and Design
Spring 2026

I. Introduction

This document defines the requirements for a web-based CRUD application developed as part of the System Analysis and Design course. Students will design and implement a full-stack system using specific technologies while applying software engineering principles.

A. Scope

The system will:

- Provide a RESTful API for data operations
- Include a frontend interface for user interaction
- Support full CRUD operations
- Be version-controlled and tested

You are free to choose the application domain and the project.

II. Overall Description

The system consists of:

- **Frontend**: Vanilla Javascript (no framework like react, etc)
- **Backend**: Node.js with Express
- **Data Layer**: You must have a database, but you are free to choose DBMS.
- **API Layer**: RESTful endpoints

III. Functional Requirements

You must define their own domain-specific features, but at minimum you need to support the following.

A. CRUD operations

Every project must support creating, reading, updating and deleting an entity.

B. REST API Requirements

Every project must use standard HTTP methods, alongside proper status codes. You must use JSON request/response format.

C. Frontend Requirements

The frontend should be implemented as a single-page application (SPA) using vanilla Javascript. The page must dynamically update content using Javascript and communicate with the backend via asynchronous API calls (fetch) without requiring full page reloads.

You are prohibited to use Javascript frameworks such as React, Vue, Angular, etc.

D. Validation

Make sure you are validating all inputs in both frontend and backend.

IV. Non-Functional Requirements

A. Code Quality

Your business logic should not live in the routes. If you write your program like that, you will not be able to unit test them. In order to unit test them, you must write your business logic separately.

B. Version Control

You must use Git and Github. Connect your Github account to your machine and push your code regularly. As a bonus, you can try to learn about Github Actions and use a basic linter to make an introduction to Devops.

C. Testing

You must do unit testing for the business logic functions. You do not need to test the routes to see if they work.

D. Documentation

Each project must include a comprehensive **README.md**. This file should detail the system setup, execution steps, and API usage. It should include information on how to reproduce your program.

V. API Documentation

The system must include an interactive API documentation using Swagger. Swagger UI must allow users to explore and test API endpoints interactively.

VI. Project Ideas

A. Topic Selection

You are free to choose the application domain. However, the project must be data-centric and suitable to demonstrate CRUD operations. Try to find a topic which you can also use in your daily life.

You are encouraged to choose a project that is personally meaningful, useful in daily life or aligned with your interests. The goal is to build a system that you would realistically use or find engaging.

These can be examples for you:

- Managing personal tasks, habits, or schedules
- Tracking expenses, goals, activities
- Organizing collections (books, movies, games, etc)
- Supporting a hobby (fitness, learning, content creation, etc)

B. Scope Constraints

To ensure appropriate complexity, the project must include at least one main entity (e.g. task, product, user, etc). It should support full CRUD operations and include at least one additional feature such as searching, simple relationships such as (categories, users, etc).

The project should not be overly simplistic. It should not be a single form with no meaningful logic.

C. Prohibited Topics

Following topics are discouraged to avoid unnecessary complexity:

- Real-time systems (chat apps, etc)
- AI-heavy or unrelated integrations
- Pure UI-focused apps without meaningful backend logic

VII. Evaluation Criteria

TABLE I
PROJECT EVALUATION CRITERIA

Criterion	**Weight (%)**
Functionality (CRUD completeness)	25
Code Quality and Modularity	20
API Design and REST Compliance	15
Swagger/OpenAPI Documentation	10
Testing	15
General Documentation (README, Setup)	10
Version Control (Git Usage)	5
Total	**100**

VIII. Deadline

There will be a section in UZEM for you to upload everything you have as a ZIP file. You are going to present your work to me on May 22 and June 5. The last day for you to upload your work as a ZIP file is May 21.

There is also a Google Sheets where you write your name, your project name and Github link.

Late submissions are subject to serious penalties.